



Internet of Things Fundamentals

Name	Kanza Kashaf
Registration NO:	22-NTU-CS-1350
Program	BSAI
Semester	6 th
Department	Computer Science
Submitted to	Sir Nasir Mahmood

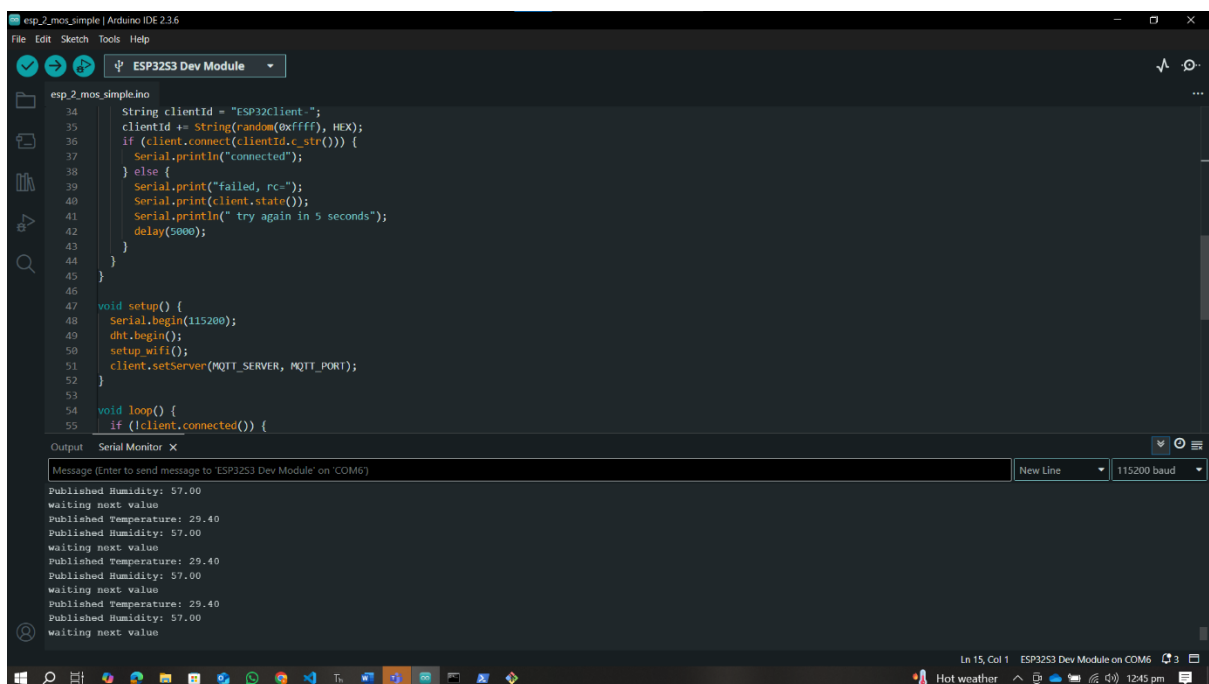
Title:

Lab 13

Task 1:

Run the Arduino-based code to **publish DHT sensor data to the Mosquitto MQTT broker.**

Here I run the Arduino code to get data from sensors and publish that data to MQTT broker. The ESP32 connects to Wi-Fi and publishes data every second. The serial monitor is showing live updates.



```
esp_2_mos_simpleino
34 String clientId = "ESP32Client-";
35 clientId += String(random(0xffff), HEX);
36 if (client.connect(clientId.c_str())) {
37   Serial.println("connected");
38 } else {
39   Serial.print("failed, rc=");
40   Serial.print(client.state());
41   Serial.println(" try again in 5 seconds");
42   delay(5000);
43 }
44 }
45 }
46
47 void setup() {
48   Serial.begin(115200);
49   dht.begin();
50   setup_wifi();
51   client.setServer(MQTT_SERVER, MQTT_PORT);
52 }
53
54 void loop() {
55   if (!client.connected()) {
```

Output Serial Monitor X

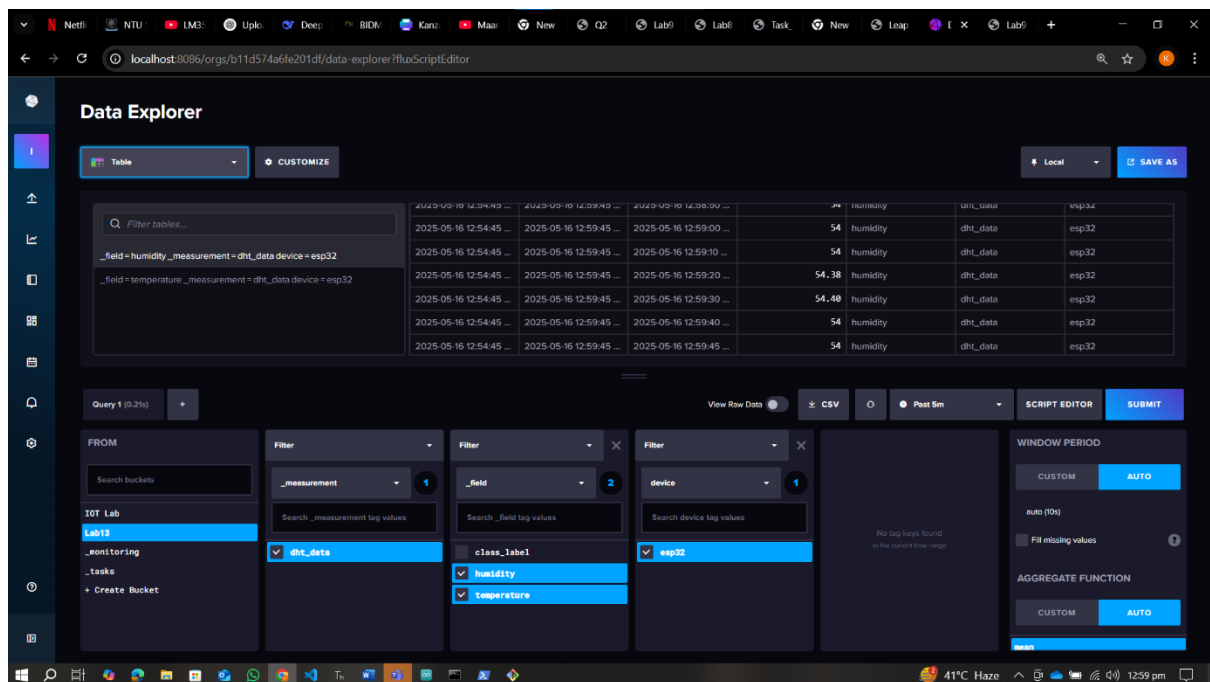
Message (Enter to send message to 'ESP3253 Dev Module' on 'COM6')

Published Humidity: 57.00
waiting next value
Published Temperature: 29.40
Published Humidity: 57.00
waiting next value
Published Temperature: 29.40
Published Humidity: 57.00
waiting next value
Published Temperature: 29.40
Published Humidity: 57.00
waiting next value

Ln 15, Col 1 ESP3253 Dev Module on COM6 3
Hot weather 12:45 pm

Here on the PowerShell, I check that my Moquitto MQTT broker is running successfully and get the real time data.

Here the data on the Dashboard of Influx DB is going successfully.



Task 3:

Run `2-train_model_with_noise.py` and **record the confusion matrix and classification report.**

Now I train our model by running that code to create a synthetic dataset and Create a neural network to classify temperature/humidity into 5 classes. And the model record confusion matrix and classification report.

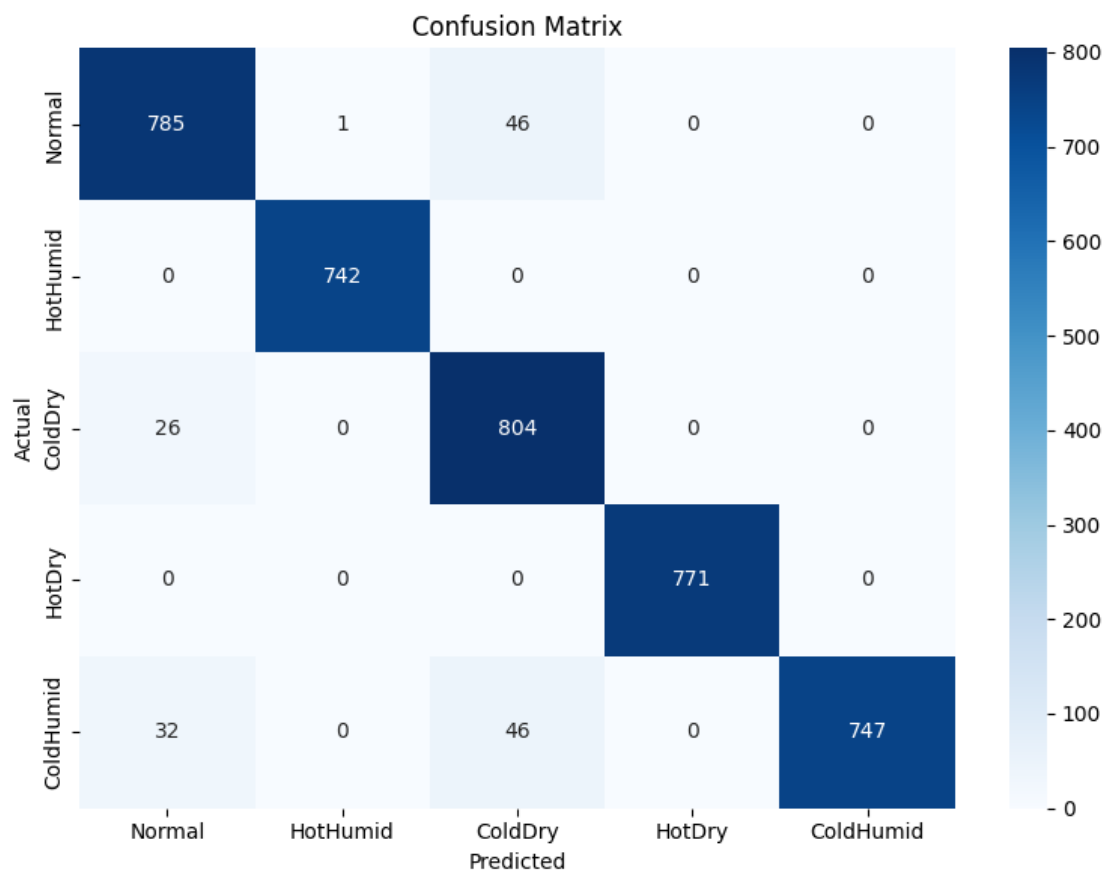
```
File Edit Selection View Go Run Terminal Help src
EXPLORER
  SRC
    > mode_modules
    > dht_classifier.js
    > index.html
    > input.css
    > normalization.rpz
    > output.css
    > package-lock.json
    > package.json
    > Task_1.html
    > Task_2.html
    > Task_3.html
  OUTLINE
  TIMELINE
  PYMAKER PROJECTS

C:\Users\BusinessTech> OneDrive - Higher Education Commission > 6th Sem > Internet of Things Fundamentals > Lab > Lab 3 > Lab13_IoT_gateway_part1 > python-scripts > 2-train_model_with_noise.py
1 # with confusion matrix and classification report
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Epoch 1/10
500/500 2s 2ms/step - accuracy: 0.4364 - loss: 1.3640 - val_accuracy: 0.9060 - val_loss: 0.6026
Epoch 2/10
500/500 1s 2ms/step - accuracy: 0.7755 - loss: 0.6857 - val_accuracy: 0.9585 - val_loss: 0.3217
Epoch 3/10
500/500 1s 2ms/step - accuracy: 0.8516 - loss: 0.4862 - val_accuracy: 0.9545 - val_loss: 0.2507
Epoch 4/10
500/500 1s 2ms/step - accuracy: 0.8728 - loss: 0.4212 - val_accuracy: 0.9590 - val_loss: 0.2207
Epoch 5/10
500/500 2s 4ms/step - accuracy: 0.8954 - loss: 0.3814 - val_accuracy: 0.9617 - val_loss: 0.2027
Epoch 6/10
500/500 1s 2ms/step - accuracy: 0.8990 - loss: 0.3622 - val_accuracy: 0.9603 - val_loss: 0.1927
Epoch 7/10
500/500 2s 3ms/step - accuracy: 0.9124 - loss: 0.3280 - val_accuracy: 0.9620 - val_loss: 0.1824
Epoch 8/10
500/500 1s 3ms/step - accuracy: 0.9135 - loss: 0.3166 - val_accuracy: 0.9615 - val_loss: 0.1761
Epoch 9/10
500/500 1s 2ms/step - accuracy: 0.9235 - loss: 0.2976 - val_accuracy: 0.9620 - val_loss: 0.1680
Epoch 10/10
500/500 1s 2ms/step - accuracy: 0.9202 - loss: 0.2898 - val_accuracy: 0.9622 - val_loss: 0.1640
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy. We recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
125/125 0s 2ms/step

Classification Report:
precision recall f1-score support
0 0.9312 0.9435 0.9373 832
1 0.9987 1.0000 0.9993 742
2 0.8973 0.9687 0.9316 830
3 1.0000 1.0000 1.0000 771
4 1.0000 0.9935 0.9967 825

accuracy 0.9623 4000
macro avg 0.9654 0.9635 0.9637 4000
weighted avg 0.9641 0.9623 0.9624 4000
```

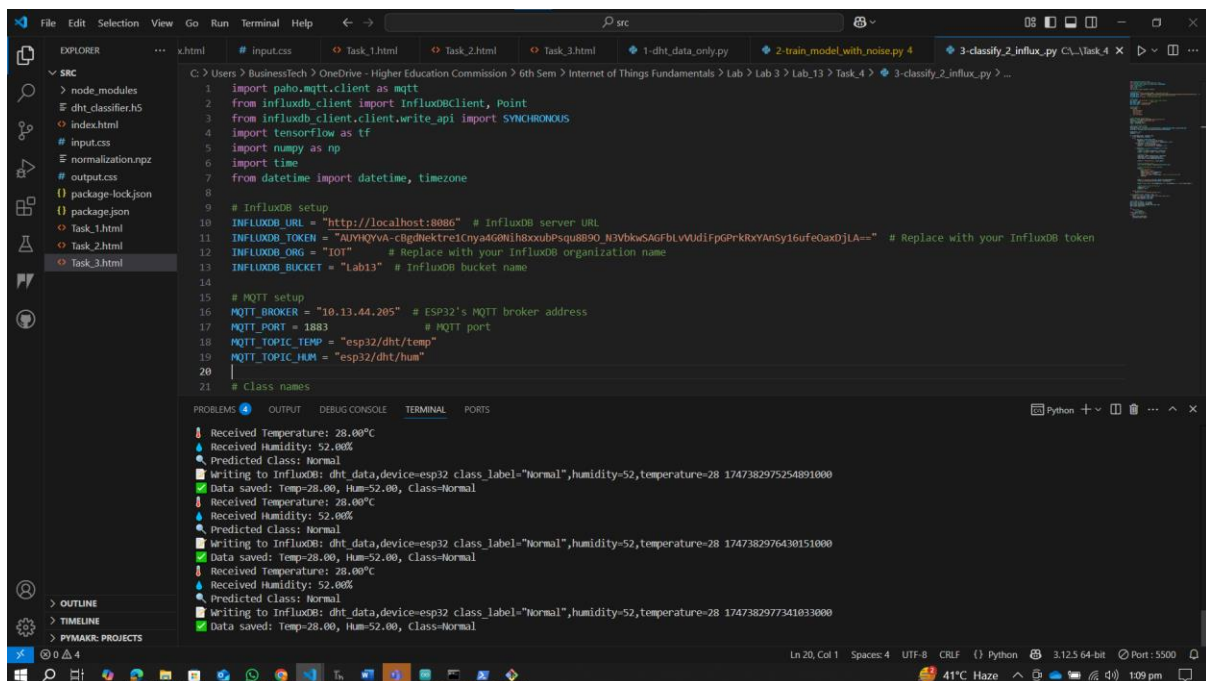
Here the Confusion Matrix of the model which I train. Showing the highest miss classification are done in Cold Humidity Class then Normal Class and so on.



Task 4:

Execute 3-classify_2_influx.py and **verify InfluxDB data** for temperature, humidity, and classification results.

Now I run this data to read live sensor data, classify it using the trained model, and save that data. The terminal is showing real time data.



The screenshot shows a VS Code editor with a Python script named `3-classify_2_influx.py` open. The script imports `paho.mqtt.client`, `InfluxDBClient`, `Point`, `tensorflow`, `numpy`, `time`, and `datetime`. It sets up InfluxDB and MQTT connections. The terminal output shows real-time sensor data being received and classified. The data is saved to InfluxDB with the class label 'Normal'.

```
1 import paho.mqtt.client as mqtt
2 from influxdb_client import InfluxDBClient, Point
3 from influxdb_client.client.write_api import SYNCHRONOUS
4 import tensorflow as tf
5 import numpy as np
6 import time
7 from datetime import datetime, timezone
8
9 # InfluxDB setup
10 INFLUXDB_URL = "http://localhost:8086" # InfluxDB server URL
11 INFLUXDB_TOKEN = "AUYHqYVA-cBgDnEktre1Cny4G6Nih8xxubPsq8B90_N3VbkwSAGfblVVUdIfGPfKkxYsY16ufe0axDjLA==" # Replace with your InfluxDB token
12 INFLUXDB_ORG = "IOT" # Replace with your InfluxDB organization name
13 INFLUXDB_BUCKET = "Lab13" # InfluxDB bucket name
14
15 # MQTT setup
16 MQTT_BROKER = "10.13.44.205" # ESP32's MQTT broker address
17 MQTT_PORT = 1883 # MQTT port
18 MQTT_TOPIC_TEMP = "esp32/dht/temp"
19 MQTT_TOPIC_HUM = "esp32/dht/hum"
20
21 # Class names
```

Terminal Output:

```
Received Temperature: 28.00°C
Received Humidity: 52.00%
Predicted Class: Normal
Writing to InfluxDB: dht_data,device-esp32 class_label="Normal",humidity=52,temperature=28 1747382975254891000
Data saved: Temp=28.00, Hum=52.00, Class=Normal
Received Temperature: 28.00°C
Received Humidity: 52.00%
Predicted Class: Normal
Writing to InfluxDB: dht_data,device-esp32 class_label="Normal",humidity=52,temperature=28 1747382976430151000
Data saved: Temp=28.00, Hum=52.00, Class=Normal
Received Temperature: 28.00°C
Received Humidity: 52.00%
Predicted Class: Normal
Writing to InfluxDB: dht_data,device-esp32 class_label="Normal",humidity=52,temperature=28 1747382977341033000
Data saved: Temp=28.00, Hum=52.00, Class=Normal
```

Here I check the data on the Dashboard of Influx DB is going successfully.

Here the class_label is added as it shows the lable of the class as I used the train model to predict the class.

[illegible]